LAPORAN PRAKTIKUM

STRUKTUR DATA

“STACK”

disusun Oleh:

NIDA TSABITA ARIBA

2511532025

Dosen Pengampu:

Dr. WAHYUDI, S.T., M.T.

Asisten Praktikum:

JOVANTRI IMMANUEL GULO

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Alhamdulillahi rabbil ‘alamin, segala puji dan syukur penulis panjatkan ke hadirat Allah SWT atas limpahan rahmat, nikmat, dan hidayah-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan judul “STACK”.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Struktur Data. Laporan ini diharapkan dapat memperdalam pemahaman penulis mengenai penerapan konsep struktur data, khususnya *Stack* (*LIFO*) dalam bahasa pemrograman Java, serta implementasinya dalam pengelolaan data.

Penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, serta asisten praktikum yang telah memberikan bimbingan dan arahan selama pelaksanaan kegiatan praktikum. Ucapan terima kasih juga disampaikan kepada teman-teman praktikan yang telah memberikan bantuan, dukungan, serta kesempatan berdiskusi dalam menyelesaikan tugas praktikum ini.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan dan keterbatasan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca.

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN 3
2.1 Program *StackArray* 3
2.2 Program *stackPostfix* 9
2.3 Program *NilaiMaximum* 12
2.4 Program *SiswaStack* 14
BAB III KESIMPULAN 18
3.1 Kesimpulan 18
DAFTAR PUSTAKA 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pembelajaran pemrograman Java, mahasiswa tidak hanya mempelajari dasar seperti perulangan dan percabangan, tetapi juga mempelajari cara mengelola data secara terstruktur. Salah satu konsep penting dalam pengelolaan data adalah penggunaan struktur data, salah satunya yaitu *Stack*.

Stack merupakan struktur data yang bekerja dengan prinsip *LIFO* (*Last In First Out*), dimana data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Dalam bahasa pemrograman Java, *Stack* dapat diimplementasikan menggunakan *class Stack* maupun menggunakan struktur lain seperti *array* atau *ArrayList*.

Pada praktikum ini, *Stack* digunakan dalam beberapa program seperti operasi dasar *stack* (*push*, *pop*, *peek*), evaluasi ekspresi *postfix*, serta pencarian nilai maksimum. Melalui praktikum ini, mahasiswa dapat memahami bagaimana cara kerja *stack* dalam mengelola data secara bertumpuk dan terstruktur.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data *Stack* di Java.
2. Mengetahui prinsip kerja *Stack* (*LIFO*).
3. Mempelajari operasi dasar *Stack* seperti *push*, *pop*, dan *peek*.
4. Menerapkan *Stack* dalam berbagai kasus seperti evaluasi *postfix* dan pencarian nilai maksimum.
5. Melatih kemampuan dalam membuat program berbasis struktur data *Stack*.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah:

1. Menambah pemahaman tentang penggunaan struktur data *Stack* di Java.
2. Membantu memahami cara kerja pengelolaan data dengan konsep *LIFO*.
3. Melatih kemampuan dalam mengimplementasikan operasi dasar *Stack*.
4. Membiasakan penggunaan *Stack* dalam pemecahan masalah pemrograman.
5. Meningkatkan kemampuan logika dalam membuat program berbasis struktur data.

BAB II

PEMBAHASAN

2.1 Program *StackArray*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan3_2511532025;

public class stackArray_2511532025 {
    static final int MAX_2025 = 1000;
    int top_2025;
    int a_2025[] = new int[MAX_2025];
    boolean isEmpty_2025()
    {
            return (top_2025 < 0);
    }
    stackArray_2511532025()
    {
            top_2025 = -1;
    }
    boolean push_2025(int x)
    {
            if (top_2025 >= (MAX_2025 -1)) {
                    System.out.println("Stack Overflow");
                    return false;
            }
            else {
                    a_2025[++top_2025] = x;
                    System.out.println(x + " dimasukkan dalam stack");
                    return true;
            }
    }
    int pop_2025()
    {
            if (top_2025 < 0) {
                    System.out.println("Stack Underflow");
                    return 0;
            }
            else {
                    int x = a_2025[top_2025--];
                    return x;
            }
    }
    int peek_2025()
    {
            if (top_2025 < 0) {
                    System.out.println("Stack Underflow");
                    return 0;
            }
            else {
                    int x = a_2025[top_2025];
                    return x;
            }
    }
    void print_2025() {
```

```
            for (int i = top_2025; i>-1; i--) {
                System.out.print(" " + a_2025 [i]);
            }
    }
}
```

Kode Program 2.1 Program *StackArray*

```
package pekan3_2511532025;

public class stackArrayDriver_2511532025 {

    public static void main(String[] args) {
            stackArray_2511532025 s = new stackArray_2511532025();
            s.push_2025(10);
            s.push_2025(20);
            s.push_2025(30);
            System.out.println(s.pop_2025() + " dikeluarkan dari stack");
            System.out.println("Elemen teratas adalah :" + s.peek_2025());
            System.out.print("Elemen pada stack :");
            s.print_2025();
    }

}
```

Kode Program 2.2 Program *StackArrayDriver*

Kode di atas berfungsi untuk menunjukkan penggunaan struktur data *Stack* dalam bahasa Java yang diimplementasikan menggunakan *array*. Program ini terdiri dari dua *class*, yaitu *class stackArray_2511532025* yang berfungsi sebagai pengelola data (menyimpan dan memanipulasi *stack*), serta *class stackArrayDriver_2511532025* yang berfungsi sebagai *driver* atau program utama untuk menjalankan dan menguji *method* yang telah dibuat.

Program ini digunakan untuk menyimpan dan mengelola sekumpulan data berupa angka (*integer*) dengan prinsip *LIFO* (*Last In First Out*), yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Beberapa operasi yang dapat dilakukan antara lain menambahkan elemen ke dalam *stack* (*push*), menghapus

elemen teratas (*pop*), melihat elemen teratas tanpa menghapusnya (*peek*), serta menampilkan seluruh isi *stack* menggunakan perulangan.

2.1.1 Langkah Kerja Program *StackArray*

1. Menentukan kapasitas maksimum *stack*

```
static final int MAX_2025 = 1000;
```

Kode Program 2.3 Kapasitas *Stack*

Baris ini digunakan untuk menentukan jumlah maksimum elemen yang dapat disimpan dalam *stack* yaitu sebanyak 1000 data.

2. Deklarasi variabel *stack*

```
int top_2025;
int a_2025[] = new int[MAX_2025];
```

Kode Program 2.4 Deklarasi *Stack*

Variabel *top_2025* digunakan untuk menunjukkan posisi elemen teratas, sedangkan *array a_2025* digunakan untuk menyimpan data *stack*.

3. Insialiasi *stack* (*konstruktor*)

```
stackArray_2511532025()
{
   top_2025 = -1;
}
```

Kode Program 2.5 Konstruktor

Kontruktor ini digunakan untuk menginisialisasi *stack* dalam keadaan kosong dengan nilai *top = -1*.

4. Mengecek apakah stack kosong

```
boolean isEmpty_2025()
{
   return (top_2025 < 0);
}
```

Kode Program 2.6 Cek *Stack* kosong

Method ini digunakan untuk mengecek apakah stack kosong atau tidak.

5. Menambahkan elemen (*push*)

```
boolean push_2025(int x)
{
   if (top_2025 >= (MAX_2025 -1)) {
            System.out.println("Stack Overflow");
            return false;
   }
   else {
            a_2025[++top_2025] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
   }
}
```

Kode Program 2.7 *Push*

Method ini digunakan untuk menambahkan elemen ke dalam *stack*. Jika *stack* penuh (*overflow*), maka akan menampilkan pesan *error*.

6. Menghapus elemen (*pop*)

```
int pop_2025()
{
   if (top_2025 < 0) {
            System.out.println("Stack Underflow");
            return 0;
   }
   else {
            int x = a_2025[top_2025--];
            return x;
   }
}
```

Kode Program 2.8 *Pop*

Method ini digunakan untuk menghapus elemen paling atas dari *stack*. Jika *stack* kosong, akan terjadi kondisi *underflow*.

7. Melihat elemen teratas (*peek*)

```
int peek_2025()
{
   if (top_2025 < 0) {
            System.out.println("Stack Underflow");
            return 0;
   }
   else {
            int x = a_2025[top_2025];
            return x;
   }
}
```

Kode Program 2.9 *Peek*

Method ini digunakan untuk melihat elemen paling atas tanpa menghapusnya.

8. Menampilkan isi *stack*

```
void print_2025() {
   for (int i = top_2025; i>-1; i--) {
            System.out.print(" " + a_2025 [i]);
   }
}
```

Kode Program 2.10 *Print Stack*

Method ini digunakan untuk menampilkan seluruh isi *stack* dari elemen teratas ke elemen terbawah.

2.1.2 Langkah Kerja Program StackArrayDriver

1. Membuat objek *stack*

```
  stackArray_2511532025 s = new
stackArray_2511532025();
  System.out.print("Elemen pada stack :");
```

Digunakan untuk membuat *stack* kosong.

2. Menambahkan elemen

```
  s.push_2025(10);
  s.push_2025(20);
  s.push_2025(30);
```

Data dimasukkan ke *stack* secara berurutan.

3. Menghapus elemen teratas

```
  System.out.println(s.pop_2025() + " dikeluarkan dari
stack");
```

Elemen terakhir (30) akan keluar terlebih dahulu (*LIFO*).

4. Menampilkan elemen teratas

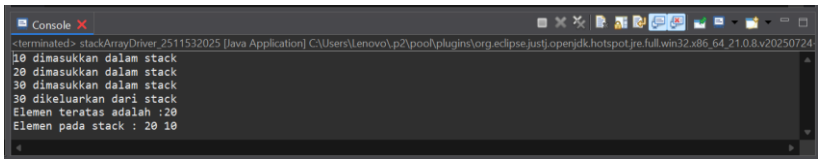
```
  System.out.println("Elemen teratas adalah :" +
s.peek_2025());
```

Menampilkan elemen paling atas tanpa menghapusnya.

5. Menampilkan seluruh isi *stack*

```
s.print_2025();
```

2.1.3 Hasil Program



Gambar 2.1 Hasil Program *StackArray*

Berdasarkan hasil eksekusi program, *stack* awalnya dalam keadaan kosong, kemudian dilakukan penambahan elemen menggunakan *method push()* yaitu 10, 20, dan 30. Hal ini menunjukkan bahwa *stack* dapat menyimpan data sesuai urutan penambahannya, dimana setiap elemen baru akan ditempatkan di posisi paling atas.

Setelah dilakukan operasi *pop()*, elemen terakhir yang dimasukkan yaitu 30 berhasil dikeluarkan dari stack. Hal ini sesuai dengan konsep *LIFO* (*Last In First Out*), dimana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Setelah elemen dihapus, posisi elemen teratas berubah menjadi 20.

Program juga menampilkan elemen teratas menggunakan *method peek()*. *Method* ini berfungsi untuk melihat elemen paling atas tanpa menghapusnya dari *stack*. Selain itu, program menampilkan seluruh isi *stack* menggunakan *method print()* dengan perulangan, sehingga elemen ditampilkan dari atas ke bawah yaitu 20 dan 10.

Secara keseluruhan, hasil program menunjukkan bahwa operasi dasar *stack* seperti *push*, *pop*, *peek*, dan print telah berjalan dengan baik sesuai dengan teori struktur data *stack*.

2.2 Program *stackPostfix*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan3_2511532025;
import java.util.Scanner;
import java.util.Stack;
public class stackPostfix_2511532025 {

        public static int postfixEvaluate_2025(String
expression) {
                Stack<Integer> s_2025 = new Stack<Integer>();
                Scanner input_2025 = new Scanner(expression);
                while (input_2025.hasNext()) {
                        if (input_2025.hasNextInt()) {

   s_2025.push(input_2025.nextInt());
                } else {
                        String operator_2025 =
input_2025.next();
                        int operand2_2025 = s_2025.pop();
                        int operand1_2025 = s_2025.pop();
                        if (operator_2025.equals("+")) {
                                s_2025.push(operand1_2025 +
operand2_2025);
                        } else if (operator_2025.equals("-")) {
                                s_2025.push(operand1_2025 -
operand2_2025);
                        } else if (operator_2025.equals("*")) {
                                s_2025.push(operand1_2025 *
operand2_2025);
                        } else {
                                s_2025.push(operand1_2025 /
operand2_2025);
                        }
                }
        }
        input_2025.close();
        return s_2025.pop();
   }
   public static void main(String[] args) {
        System.out.println("hasil postfix = " +
postfixEvaluate_2025("5 2 4 * + 7 -"));
   }
}
```

Kode di atas berfungsi untuk menunjukkan penggunaan struktur data *Stack* dalam bahasa Java untuk mengevaluasi ekspresi *postfix*. Program ini menggunakan *class Stack* dari *Java Collection* dan *class Scanner* untuk membaca setiap elemen dari ekspresi. Program terdiri dari satu *class* yaitu *stackPostfix_2511532025* yang berfungsi untuk memproses dan menghitung hasil dari ekspresi *postfix*.

Program ini digunakan untuk menghitung ekspresi matematika dalam bentuk *postfix*, dimana operator ditulis setelah *operand*. Beberapa operasi yang dilakukan antara lain membaca angka, menyimpan ke dalam *stack*, mengambil dua *operand* saat menemukan operator, melakukan perhitungan, dan menyimpan kembali hasilnya ke dalam *stack* hingga diperoleh hasil akhir.

2.2.1 Langkah Kerja Program *StackPostfix*

1. *Import Library*
   Digunakan untuk menggunakan class *Stack* sebagai struktur data dan *Scanner* untuk membaca ekspresi.
2. Membuat *method evaluasi postfix*
   Method postfixEvaluate_2025() digunakan untuk menghitung hasil dari ekspresi *postfix*.
3. Membuat *stack*
   Stack digunakan untuk menyimpan *operand* (angka) sementara selama proses perhitungan.
4. Inisialisasi *Scanner*
   Scanner dibuat dari *string* ekspresi untuk membaca setiap elemen (angka atau operator).
5. Proses perulangan
   Perulangan dilakukan selama masih ada elemen dalam ekspresi menggunakan *while (input_2025.hasNext()).*

6. Jika elemen adalah angka

   Angka langsung dimasukkan ke dalam *stack* menggunakan *method push()*.

7. Jika elemen adalah operator
   - Mengambil dua *operand* dari *stack* (*operand2* diambil lebih dulu, lalu *operand1*)
   - Melakukan operasi sesuai operator (+, -, *, /)
   - Menyimpan hasil perhitungan kembali ke dalam *stack* menggunakan *push()*

8. Menutup *Scanner*

   Scanner ditutup menggunakan *input_2025.close()* untuk menghindari kebocoran *resource*.

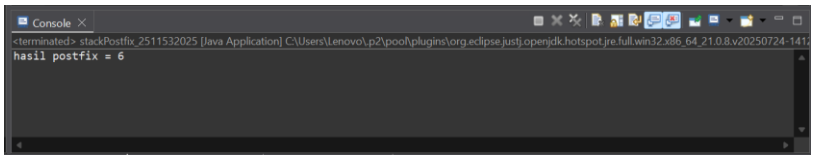
9. Mengembalikan hasil akhir

   Hasil akhir diambil dari *stack* menggunakan *pop()* setelah semua ekspresi diproses.

10. Menjalankan program

    Method dipanggil di *main()* dengan input ekspresi *postfix*, kemudian hasil ditampilkan.

2.2.2 Hasil Program



Gambar 2.2 Hasil Program *StackPostfix*

Berdasarkan hasil eksekusi program, ekspresi *postfix* "5 2 4 * + 7 -" berhasil dihitung menggunakan struktur data *stack*. Proses dimulai dengan memasukkan angka ke dalam *stack* hingga ditemukan operator. Saat operator ditemukan, dua elemen terakhir diambil dari stack untuk dilakukan operasi.

Perhitungan dilakukan secara bertahap, yaitu:

- $2 \times 4 = 8$

- 5 + 8 = 13
- 13 - 7 = 6

Hasil akhir yang diperoleh adalah 6, yang menunjukkan bahwa algoritma *postfix* berjalan dengan benar sesuai konsep *stack*.

Program ini menunjukkan bahwa struktur data *stack* sangat efektif digunakan dalam proses evaluasi ekspresi *postfix* karena mengikuti prinsip *LIFO* (*Last In First Out*), dimana operand terakhir yang masuk akan diproses terlebih dahulu.

2.3 Program *NilaiMaximum*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan3_2511532025;
import java.util.Stack;
public class NilaiMaximum_2511532025 {
   public static int max_2025 (Stack<Integer> s_2025) {
         Stack<Integer> backup = new Stack<Integer>();
         int maxValue = s_2025.pop();
         backup.push(maxValue);
         while (!s_2025.isEmpty()) {
               int next = s_2025.pop();
               backup.push(next);
               maxValue = Math.max(maxValue,  next);
         }
         while (!backup.isEmpty()) {
               s_2025.push(backup.pop());
         }
         return maxValue;
   }
   public static void main(String[] args) {
   Stack<Integer> s_2025 = new Stack<Integer>();
   s_2025.push(70);
   s_2025.push(12);
   s_2025.push(20);
   System.out.println("isi stack " + s_2025);
   System.out.println("Stack Teratas " + s_2025.peek());
   System.out.println("Nilai maksimum " + max_2025(s_2025));
   }

}
```

2.3.1 Langkah Kerja Program *NilaiMaximum*

1. *Import Stack*

   Digunakan untuk menggunakan struktur data *stack*.

2. Membuat *method max*

   Method max_2025() digunakan untuk mencari nilai maksimum dari elemen dalam *stack*.

3. Membuat *stack backup*

   Digunakan untuk menyimpan sementara elemen agar data asli tidak hilang.

4. Mengambil elemen pertama

   Elemen paling atas diambil menggunakan *pop()* dan dijadikan nilai awal maksimum (*maxValue*), kemudian disimpan ke *stack backup*.

5. Perulangan untuk memproses seluruh elemen

   Perulangan dilakukan selama *stack* belum kosong menggunakan *while (!s_2025.isEmpty()).*

6. Menyimpan ke *backup*

   Setiap elemen yang diambil dimasukkan ke *stack backup.*

7. Mengembalikan isi *stack*

   Setelah semua elemen diproses, isi *stack* dikembalikan seperti semula dengan memindahkan data dari *stack* backup ke *stack* awal.

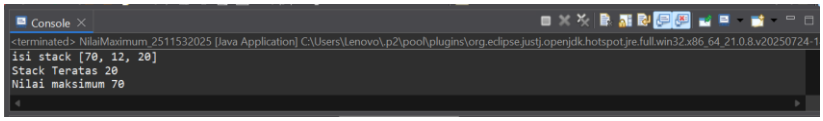
8. Mengembalikan nilai maksimum

   Nilai maksimum yang telah diperoleh dikembalikan sebagai hasil *method*.

9. Menampilkan hasil

   Pada *method main(),* program menampilkan isi *stack*, elemen teratas menggunakan *peek()*, dan\ nilai maksimum yang diperoleh.

2.3.2 Hasil Program



Berdasarkan hasil eksekusi program, *stack* berisi elemen [70, 12, 20] yang dimasukkan menggunakan *method push()*. Elemen

teratas dari *stack* adalah 20, yang ditampilkan menggunakan *method peek()*.

Program kemudian mencari nilai maksimum menggunakan *method max_2025()*. Proses dilakukan dengan mengambil setiap elemen dari *stack*, membandingkannya menggunakan *Math.max()*, dan menyimpan sementara ke dalam *stack* lain. Setelah seluruh elemen diperiksa, diperoleh nilai maksimum yaitu **70**.

Selain itu, setelah proses pencarian selesai, isi *stack* tetap kembali seperti semula yaitu [70, 12, 20]. Hal ini menunjukkan bahwa program berhasil mencari nilai maksimum tanpa merusak data asli dalam *stack*.

Secara keseluruhan, program ini menunjukkan bahwa struktur data stack dapat digunakan tidak hanya untuk penyimpanan data, tetapi juga untuk proses pencarian nilai tertentu dengan bantuan stack tambahan.

2.4 Program *SiswaStack*

Kode program yang dipraktikkan adalah sebagai berikut:

```
package pekan3_2511532025;
import java.util.ArrayList;

class Siswa_2511532025 {
   String nama_2025;
   int nim_2025;

   public Siswa_2511532025(String nama_2025, int nim_2025) {
         this.nama_2025 = nama_2025;
         this.nim_2025 = nim_2025;
   }
   @Override
   public String toString() {
         return "Nim: " + nim_2025 + ", Nama: " + nama_2025;
   }
}

public class SiswaStack_2511532025 {
   private ArrayList<Siswa_2511532025> stack;
   public SiswaStack_2511532025() {
         stack = new ArrayList<>();
   }
```

```java
public void push(Siswa_2511532025 mhs) {
        stack.add(mhs);
    }

public Siswa_2511532025 pop_2025() {
    if (!isEmpty()) {
        return stack.remove(stack.size() - 1);
    }
    return null;
}
public Siswa_2511532025 peek_2025() {
    if (!isEmpty()) {
        return stack.get(stack.size() - 1);
    }
    return null;
}
public boolean isEmpty() {
    return stack.isEmpty();
}
public void tampilkanSiswa_2025() {
    for (int i = stack.size() - 1; i >= 0; i--) {
        System.out.println(stack.get(i));
    }
}


public static void main (String[] args) {
    SiswaStack_2511532025 studentStack_2025 = new
SiswaStack_2511532025();

    Siswa_2511532025 mhs1 = new Siswa_2511532025 ("Ali", 1);
    Siswa_2511532025 mhs2 = new Siswa_2511532025 ("Boby", 2);
    Siswa_2511532025 mhs3 = new Siswa_2511532025 ("Charles",
3);

    studentStack_2025.push(mhs1);
    studentStack_2025.push(mhs2);
    studentStack_2025.push(mhs3);

    System.out.println("Siswa di  dalam stack:");
    studentStack_2025.tampilkanSiswa_2025();

    System.out.println("Siswa teratas " +
studentStack_2025.peek_2025());
    System.out.println("mengeluarkan siswa teratas dari stack:
" + studentStack_2025.pop_2025());
    studentStack_2025.tampilkanSiswa_2025();
}
}
```

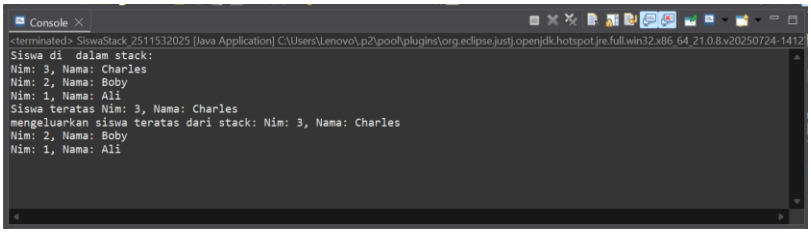
Program di atas berfungsi untuk menunjukkan penggunaan struktur data *Stack* menggunakan *ArrayList* dalam bahasa Java. Program ini terdiri dari dua *class* yaitu *Siswa_2511532025* sebagai representasi data

mahasiswa (nama dan NIM), serta *SiswaStack_2511532025* sebagai implementasi struktur *stack*.

Stack diimplementasikan menggunakan *ArrayList*, dimana elemen terakhir dianggap sebagai elemen teratas (*top*). Program ini menyediakan operasi dasar *stack* seperti *push*, *pop*, dan *peek*, serta menampilkan isi *stack*.

Program ini digunakan untuk menyimpan data siswa ke dalam *stack*, mengambil data siswa terakhir yang masuk, serta melihat elemen teratas tanpa menghapusnya.

2.4.1 Hasil Program



Berdasarkan hasil eksekusi program, data siswa berhasil dimasukkan ke dalam *stack* menggunakan operasi *push*. Urutan penyimpanan mengikuti prinsip *LIFO* (*Last In First Out*), dimana data terakhir yang dimasukkan akan berada di posisi teratas.

Urutan data dalam *stack*:

- Charles (teratas)
- Boby
- Ali

Saat dilakukan operasi *peek*, program menampilkan siswa teratas yaitu Charles tanpa menghapusnya.

Kemudian saat dilakukan operasi *pop*, siswa teratas yaitu Charles dikeluarkan dari *stack*.

Setelah itu, isi *stack* menjadi:

- Boby
- Ali

Hal ini menunjukkan bahwa struktur data *stack* bekerja sesuai konsep *LIFO*, dimana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

BAB III
KESIMPULAN

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa *Stack* merupakan salah satu struktur data dalam Java yang digunakan untuk menyimpan data dengan prinsip *LIFO* (*Last In First Out*), yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan.

Melalui beberapa program yang telah dibuat, seperti implementasi *stack* menggunakan *array*, evaluasi ekspresi *postfix*, pencarian nilai maksimum, serta penyimpanan data objek, dapat diketahui bahwa *Stack* sangat membantu dalam pengelolaan data yang bersifat bertumpuk. Operasi dasar seperti menambah data (*push*), menghapus data (*pop*), dan melihat data teratas (*peek*) dapat dilakukan dengan mudah menggunakan *Stack*.

Selain itu, praktikum ini juga membantu dalam memahami bagaimana konsep *Stack* diterapkan dalam berbagai kasus, seperti perhitungan ekspresi matematika dan pengolahan data. Dengan demikian, penggunaan *Stack* sangat berguna dalam pembuatan program yang membutuhkan pengelolaan data secara terstruktur dan sistematis dalam bahasa Java.

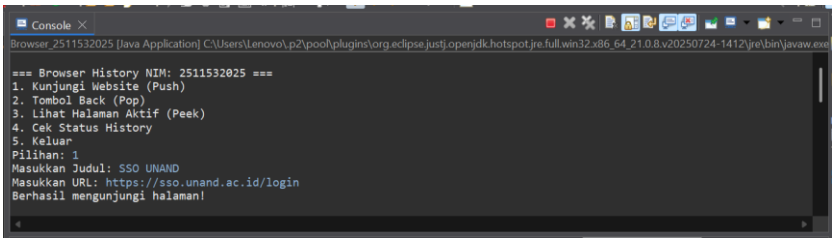
DAFTAR PUSTAKA

Wahyudi, S.T., M.T., *Struktur Data: Pekan 4 Stack*. Padang: Universitas Andalas, 2025.

Tugas Praktikum Pekan 3: Struktur Data

Topik : Implementasi *Stack* (*LIFO*) menggunakan *java.util.Stack*

Tema : Simulasi *Browser History*

1. Deskripsi Tugas

   Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.
2. Struktur Program
   1) Kelas ADT Website
   2) Kelas Driver
3. Contoh Tampilan Output Program